

Báo cáo training Data Engineer

Chương 1: Cơ sở dữ liệu - SQL Database

Họ và tên	:	Lê Anh Quân
Ngày tháng	:	07/07/2026
Người hướng dẫn	:	Thịnh

Mục lục

I. Môi trường nghiên cứu	3
II. Tổng quan về SQL	3
1. SQL là gì?	3
2. Các thành phần của SQL	3
a. DDL, DML và DCL trong SQL	3
b. Filtering, aggregation, join	4
c. Subquery & CTE	6
d. Window Functions	7
e. View	8
III. OLTP và OLAP là gì? So sánh 2 loại công nghệ xử lý dữ liệu	8
1. OLTP là gì?	8
2. OLAP là gì?	9
3. Điểm khác biệt giữa OLTP và OLAP	10
IV. ACID và CAP Theorem	10
1. ACID là gì?	10
2. CAP Theorem là gì?	11
V. Các yếu tố quan trọng trong quản trị cơ sở dữ liệu và thiết kế hệ thống	11
1. Index	11
2. Transaction	12
3. Locking	12
4. Partition	13
5. Sharding	13
6. Isolation	13
7. Connection pooling	14

I. Môi trường nghiên cứu

Ngôn ngữ: SQL

Hệ điều hành: Windows

Môi trường khác:

II. Tổng quan về SQL

1. SQL là gì?

SQL (tên đầy đủ là Structured Query Language) là ngôn ngữ truy vấn có cấu trúc, được sử dụng để giao tiếp, quản lý và thao tác với các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS). Cụm từ “truy vấn có cấu trúc” ý chỉ rằng SQL hoạt động với dữ liệu có cấu trúc (Structured Data) và cú pháp của nó phải tuân theo một cấu trúc ngữ pháp nhất định. Nó cho phép người dùng dễ dàng tạo, đọc, cập nhật và xóa dữ liệu. Các chức năng chính của SQL bao gồm:

- Truy vấn dữ liệu
- Quản lý dữ liệu
- Quản lý cấu trúc

Các hệ thống RDBMS phổ biến sử dụng SQL hiện nay bao gồm:

- MySQL
- Microsoft SQL Server
- PostgreSQL
- SQLite

2. Các thành phần của SQL

a. DDL, DML và DCL trong SQL

SQL DDL (Data Definition Language)

DDL là nhóm lệnh trong SQL dùng để xây dựng, chỉnh sửa và quản lý “bộ khung” của cơ sở dữ liệu. Các thao tác này bao gồm quản lý bảng, chỉ mục, ràng buộc và lược đồ. Các lệnh DDL cốt lõi bao gồm:

- CREATE: Tạo các đối tượng như Database, Table, View hoặc Index
- ALTER: Thay đổi cấu trúc của các đối tượng đã tồn tại (thêm cột, sửa kiểu dữ liệu, xóa ràng buộc)
- DROP: Xóa hoàn toàn đối tượng cùng toàn bộ dữ liệu và cấu trúc bên trong khỏi cơ sở dữ liệu

SQL DML (Data Manipulation Language)

DML là nhóm câu lệnh SQL được dùng để thao tác và xử lý dữ liệu trong cơ sở dữ liệu. Các lệnh DML cốt lõi bao gồm:

- **INSERT:** Dùng để thêm một hoặc nhiều bản ghi vào bảng
- **UPDATE:** Dùng để sửa đổi các bản ghi hiện có trong bảng
 - *Lưu ý luôn kèm theo điều kiện WHERE để tránh cập nhật toàn bộ dữ liệu*
- **DELETE:** Dùng để xóa các bản ghi không còn cần thiết khỏi bảng
 - *Lưu ý: Tương tự như UPDATE, nếu thiếu WHERE, toàn bộ dữ liệu trong bảng sẽ bị xóa*
- **SELECT:** Dùng để tìm kiếm và trích xuất dữ liệu từ một hoặc nhiều bảng

SQL DCL (Data Control Language)

DCL là nhóm lệnh SQL giúp quản trị viên kiểm soát ai được phép xem hoặc chỉnh sửa dữ liệu, đảm bảo an toàn hệ thống với hai lệnh:

- **GRANT:** Dùng để cấp một số quyền nhất định cho người dùng
- **REVOKE:** Dùng để thu hồi lại những quyền đã được cấp trước đó từ người dùng

b. Filtering, aggregation, join

SQL filtering (lọc dữ liệu)

Trong SQL, lọc dữ liệu là quá trình trích xuất và chỉ hiển thị các bản ghi thỏa mãn một hoặc nhiều tiêu chí cụ thể từ cơ sở dữ liệu. Quá trình này giúp loại bỏ các dữ liệu không cần thiết, cho phép tập trung vào phân tích các thông tin quan trọng. Việc lọc dữ liệu thường được thực hiện thông qua các mệnh đề và toán tử sau:

- **Mệnh đề WHERE**

Đây là mệnh đề phổ biến nhất trong SQL. Nó hoạt động như một “bộ lọc”, chỉ giữ lại những hàng mà điều kiện đánh giá là đúng.

VD: Lấy tất cả nhân viên có mức lương lớn hơn 30 triệu

```
SELECT * FROM NhanVien WHERE Luong > 30000000;
```

- **Các toán tử lọc thường dùng**
 - **Toán tử so sánh:** =, <> (khác), >, <, >=, <=
 - **AND/OR:** Dùng để kết hợp nhiều điều kiện lọc cùng lúc
 - **BETWEEN:** Lọc dữ liệu trong một khoảng giá trị
 - **IN:** Lọc các hàng có giá trị khớp với một danh sách cụ thể
 - **LIKE:** Lọc theo mẫu ký tự, thường dùng kết hợp với dấu % để tìm kiếm chuỗi

Có thể kết hợp các toán tử kể trên với mệnh đề WHERE để tạo ra các bộ lọc đa dạng.

- Mệnh đề HAVING

Được dùng chung với GROUP BY khi bạn muốn lọc trên các kết quả đã được tính toán, tổng hợp.

VD: Lấy ra các phòng ban có tổng quỹ lương lớn hơn 100 triệu

```
SELECT PhongBan, SUM(Luong)
FROM NhanVien GROUP BY PhongBan
HAVING SUM(Luong) > 100000000;
```

- Điều kiện FILTER (dành riêng cho hàm tổng hợp)

Trong một số hệ quản trị cơ sở dữ liệu (như PostgreSQL), mệnh đề FILTER cho phép bạn chỉ áp dụng hàm tính toán (như SUM(), AVG()) cho một tập hợp con các hàng đã được lọc cụ thể.

- Mệnh đề QUALIFY (lọc kết quả hàm Window)

Được sử dụng khi bạn dùng các hàm cửa sổ (Window function như ROW_NUMBER(), RANK()) và muốn lọc trực tiếp kết quả của chúng mà không cần phải bọc trong một câu lệnh SELECT khác.

SQL aggregation (tổng hợp dữ liệu)

Trong SQL, là quá trình thu thập và xử lý các giá trị từ nhiều hàng dữ liệu khác nhau để trả về một kết quả duy nhất. Quá trình này chủ yếu thực hiện thông qua các hàm tổng hợp (Aggregate Functions) để phân tích, tính toán, và tạo báo cáo. Các hàm tổng hợp (Aggregate Functions) cốt lõi bao gồm:

- **SUM:** Tính tổng giá trị một tập hợp các giá trị số
- **AVG:** Tính giá trị trung bình cộng
- **COUNT:** Đếm số lượng hàng hoặc các giá trị không bị bỏ qua (non-NULL)
- **MIN:** Tìm giá trị nhỏ nhất trong một tập hợp
- **MAX:** Tìm giá trị lớn nhất trong một tập hợp

SQL JOIN

Trong SQL, JOIN là một mệnh đề được dùng để kết nối và kết hợp dữ liệu từ hai hay nhiều bảng khác nhau dựa trên một cột (hoặc điều kiện) có điểm chung giữa các bảng đó. Các loại JOIN phổ biến bao gồm:

- INNER JOIN (Kết nối giao nhau)

Chỉ trả về các bản ghi (hàng) có giá trị khớp (tồn tại) ở cả **hai bảng**. Những hàng không khớp sẽ bị loại bỏ.

- LEFT JOIN (Kết nối bảng bên trái)

Trả về tất cả các bản ghi từ bảng bên trái và các bản ghi khớp từ bảng bên phải. Nếu không có sự khớp lệnh, kết quả trả về giá trị NULL cho các cột của bảng bên phải.

- RIGHT JOIN (Kết nối bảng bên phải)

Tương tự như LEFT JOIN, nhưng trả về tất cả các bản ghi từ bảng bên phải và các bản ghi khớp từ bảng bên trái. Những bản ghi bên phải không khớp sẽ nhận giá trị NULL cho các cột bên trái.

- FULL JOIN (Kết nối toàn bộ)

Trả về tất cả các bản ghi khi có kết quả khớp ở **một trong hai bảng**. Nếu không khớp, giá trị sẽ là NULL. (Lưu ý: Một số hệ quản trị cơ sở dữ liệu dùng cú pháp FULL OUTER JOIN).

- CROSS JOIN (Kết nối chéo)

Tạo ra tích Đề-các (Cartesian product) của hai bảng. Nó kết hợp mỗi hàng của bảng thứ nhất với tất cả các hàng của bảng thứ hai.

c. Subquery & CTE

Subquery (truy vấn con)

Subquery (truy vấn con) trong SQL là một câu lệnh truy vấn (thường là SELECT) được đặt lồng bên trong một câu lệnh SQL khác (như SELECT, INSERT, UPDATE, hoặc DELETE). Nó cho phép bạn thực hiện các bước tính toán hoặc lọc dữ liệu phức tạp dựa trên kết quả của một truy vấn khác.

- Subquery với WHERE (VD: Tìm nhân viên có mức lương cao nhất)

Truy vấn con sẽ tìm ra mức lương cao nhất, và truy vấn chính sẽ lấy ra thông tin của nhân viên nhận mức lương đó:

```
SELECT TenNhanVien, Luong
FROM NhanVien
WHERE Luong = (SELECT MAX(Luong) FROM NhanVien);
```

- Subquery với FROM (VD: Tính trung bình)

Sử dụng subquery như một bảng ảo (Inline View) để tính toán trước khi dùng truy vấn chính:

```
SELECT AVG(LuongTrungBinh)
FROM (
    SELECT PhongBanID, AVG(Luong) as LuongTrungBinh
    FROM NhanVien
    GROUP BY PhongBanID
) AS BangAo;
```

CTE (Common Table Expression)

CTE là một tập kết quả tạm thời được đặt tên trong SQL, chỉ tồn tại duy nhất trong phạm vi của câu lệnh (SELECT, INSERT, UPDATE...) ngay sau đó. Nó giúp chia nhỏ các truy vấn phức tạp thành các khối logic dễ đọc và dễ bảo trì hơn so với Subquery.

Cùng một ví dụ như phần subquery với FROM, thay vì phải viết một câu lệnh dài và lồng ghép phức tạp, CTE cho phép định nghĩa một bảng ảo (BangTam) và sử dụng lại nó ngay lập tức:

```
WITH BangTam AS (
    SELECT NhanVienID, Ten, Luong
    FROM NhanVien
    WHERE Luong > 10000000
)
SELECT *
FROM BangTam
ORDER BY Luong DESC;
```

d. Window Functions

Window functions trong SQL thực hiện các phép tính trên một tập hợp các hàng có liên quan đến hàng hiện tại. Khác với hàm tổng hợp truyền thống (như SUM() hay AVG()) gộp nhiều hàng thành một kết quả duy nhất, window function vẫn giữ nguyên từng hàng riêng lẻ và cho phép bạn xem giá trị tổng hợp bên cạnh dữ liệu gốc.

Một câu lệnh window function luôn bao gồm mệnh đề OVER(), có dạng như sau:

Hàm() OVER (PARTITION BY cột_nhóm ORDER BY cột_sắp_xếp)

Trong đó:

- **PARTITION BY:** Chia tập dữ liệu thành các phân vùng (nhóm) nhỏ để tính toán độc lập (tương tự như GROUP BY)
- **ORDER BY:** Xác định thứ tự các hàng bên trong mỗi phân vùng trước khi áp dụng hàm (mặc định là theo thứ tự tăng dần ASC)

Có tổng cộng 3 nhóm hàm phổ biến nhất: nhóm hàm tổng hợp, nhóm hàng xếp hạng và nhóm hàm phân tích.

Nhóm hàm tổng hợp (Aggregate Window Functions)

Dùng để tính tổng, trung bình, đếm, tìm min/max cho từng phân vùng.

VD: SUM(Luong) OVER(PARTITION BY PhongBan) tính tổng lương của từng phòng ban nhưng vẫn giữ nguyên chi tiết từng nhân viên.

Nhóm hàm xếp hạng (Ranking Window Functions)

Dùng để đánh số thứ tự hoặc xếp hạng các hàng trong phân vùng dựa trên một tiêu chí cụ thể.

- **ROW_NUMBER():** Đánh số thứ tự liên tục cho từng hàng (1, 2, 3...)
- **RANK():** Xếp hạng hàng, các hàng có giá trị giống nhau sẽ có cùng hạng và tạo ra khoảng trống trong thứ tự (VD: 1, 2, 2, 4)
- **DENSE_RANK():** Tương tự RANK nhưng không tạo khoảng trống (ví dụ: 1, 2, 2, 3)

Nhóm hàm phân tích (Analytic Functions)

Dùng để so sánh các hàng với nhau dựa trên vị trí của chúng.

- **LEAD():** Lấy giá trị của hàng *phía sau* hàng hiện tại.
- **LAG():** Lấy giá trị của hàng *phía trước* hàng hiện tại.

e. View

Trong SQL, View (Khung nhìn) là một bảng ảo. Thực chất, nó là một câu lệnh SELECT đã được đặt tên và lưu trữ sẵn trong cơ sở dữ liệu. View không lưu trữ dữ liệu thực tế mà chỉ chứa các hàng và cột được sinh ra từ truy vấn mỗi khi gọi đến nó. Để tạo một View cần sử dụng lệnh CREATE VIEW:

```
CREATE VIEW TenView AS
SELECT col1, col2
FROM TenBang
WHERE [cond];
```

Để sử dụng View cần phải truy vấn với cú pháp:

```
SELECT * FROM TenView
```

III. OLTP và OLAP là gì? So sánh 2 loại công nghệ xử lý dữ liệu

1. OLTP là gì?

OLTP (viết tắt cho Online Transaction Processing) là hệ thống cơ sở dữ liệu được thiết kế để quản lý, thực hiện và cập nhật các giao dịch thời gian thực với khối lượng lớn.

Các thao tác này thường ngắn, nhanh và tập trung vào dữ liệu hiện tại, ví dụ như chuyển tiền ngân hàng hay đặt vé concert. Đặc điểm cốt lõi của OLTP bao gồm:

- Xử lý thời gian thực: Phản hồi ngay lập tức khi người dùng thao tác, thời gian tính bằng ms
- Giao dịch nhỏ gọn: Mỗi thao tác thường chỉ tác động đến 1 hoặc vài hàng dữ liệu
- Tính nhất quán cao (ACID): Đảm bảo dữ liệu chính xác tuyệt đối, tránh lỗi trừ tiền hai lần hay bán một mặt hàng cho nhiều người cùng lúc

OLTP thường được sử dụng trong các mảng:

- Ngân hàng: Rút tiền tại ATM, chuyển khoản qua các app banking
- Thương mại điện tử: Thêm hàng vào giỏ, thanh toán đơn hàng
- Vận tải: Đặt vé máy bay, đặt xe công nghệ

2. OLAP là gì?

OLAP (viết tắt cho Online Analytical Processing) là hệ thống cơ sở dữ liệu được thiết kế để tối ưu việc phân tích dữ liệu lớn. OLAP giúp các nhà quản lý truy vấn, tổng hợp và phân tích dữ liệu từ nhiều góc độ khác nhau để tìm ra xu hướng và đưa ra các dự đoán. Đặc điểm cốt lõi của OLAP bao gồm:

- Tốc độ phản hồi cực nhanh: Xử lý hàng triệu dòng dữ liệu trong vài giây
- Dữ liệu lịch sử khổng lồ: Lưu trữ dữ liệu từ nhiều năm để tìm ra xu hướng
- Chỉ đọc (Read-only): Dữ liệu ít khi bị sửa đổi sau khi đã nạp vào
- Phân tích đa chiều: Nhìn nhận một con số qua nhiều yếu tố

Trọng tâm của OLAP là mô hình lập phương dữ liệu (OLAP cube). Cho dù được gọi là lập phương, trong thực tế một OLAP cube là một khối lập phương đa chiều (hypercube) với số chiều phụ thuộc vào số thuộc tính của dữ liệu. Cấu trúc dữ liệu có rất nhiều điểm tương đồng với không gian vector đa chiều được đề cập tới trong giáo trình Đại số tuyến tính, với mỗi ô (cell) trong OLAP cube là một tọa độ trong không gian vector, còn đầu của vector là một giá trị vô hướng được lưu trữ trong kho dữ liệu (data warehouse).

Các thao tác phân tích phổ biến trong OLAP bao gồm:

- Roll-up: Thu nhỏ từ chi tiết lên tổng quát (VD: Cửa hàng => Thành phố => Quốc gia)
- Drill-down: Phóng to từ tổng quát xuống chi tiết (VD: Năm => Quý => Tháng)
- Slice: Chọn cố định/giảm một chiều của OLAP cube (VD: chỉ xem dữ liệu năm 2025)

- Dice: Cắt một khối nhỏ từ khối lớn (VD: xem doanh thu của MacBook tại Hà Nội trong Quý 1 năm nay)

3. Điểm khác biệt giữa OLTP và OLAP

Điểm khác biệt cốt lõi giữa OLAP và OLTP nằm ở nguyên lý vận hành và mục đích sử dụng. OLTP, nền tảng của database, được sử dụng để ghi chép, thay đổi và truy xuất dữ liệu trong một khoảng thời gian ngắn (2-3 ms), trong khi OLAP, nền tảng của data warehouse, được sử dụng để lưu trữ, đọc và phân tích khối lượng dữ liệu khổng lồ được trải dài qua nhiều năm để hỗ trợ đưa ra quyết định.

Dưới đây là bảng so sánh chi tiết giữa hai hệ thống:

Đặc điểm	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Mục đích chính	Vận hành, ghi chép giao dịch thời gian thực	Phân tích dữ liệu lớn để đưa ra quyết định
Độ phức tạp truy vấn	Rất đơn giản, xử lý từng dòng dữ liệu cụ thể	Rất phức tạp, quét qua hàng triệu đến tỷ dòng dữ liệu
Tốc độ phản hồi	2-3 mili giây (ms)	Tùy theo khối lượng dữ liệu
Độ tuổi dữ liệu	Chỉ chứa dữ liệu hiện tại	Chứa dữ liệu lịch sử vài năm
Dung lượng lưu trữ	Nhỏ đến trung bình	Rất lớn
Người dùng chính	Nhân viên quầy, khách hàng	Nhà quản lý, DA, BA

IV. ACID và CAP Theorem

1. ACID là gì?

ACID trong SQL là tập hợp 4 thuộc tính cốt lõi của Transaction (Giao dịch) giúp đảm bảo tính toàn vẹn, chính xác và an toàn của dữ liệu, ngay cả khi hệ thống xảy ra sự cố. Nó là viết tắt của các từ tiếng anh sau:

- Atomicity (Tính nguyên tử): Mọi thao tác trong một Transaction phải thành công toàn bộ hoặc thất bại toàn bộ. Không có trường hợp chỉ được thực hiện một nửa. Nếu lỗi xảy ra, hệ thống sẽ tự động hoàn tác (rollback) về trạng thái ban đầu.

- Consistency (Tính nhất quán): Các dữ liệu được thay đổi trong Transaction phải tuân thủ nghiêm ngặt mọi quy tắc, ràng buộc, cấu trúc đã định nghĩa trước đó trong cơ sở dữ liệu
- Isolation (Tính cô lập): Nếu có nhiều Transaction cùng chạy một lúc, các giao dịch này không được phép can thiệp hoặc làm ảnh hưởng đến dữ liệu của nhau
- Durability (Tính bền vững): Một khi Transaction đã báo thành công (commit), dữ liệu sẽ được lưu trữ vĩnh viễn trên ổ cứng, không bị mất ngay cả khi xảy ra sự cố mất điện hoặc sập hệ thống

2. CAP Theorem là gì?

Định lý CAP (Định lý Brewer) trong là nguyên tắc cốt lõi trong hệ thống phân tán, phát biểu rằng một hệ thống cơ sở dữ liệu chỉ có thể đảm bảo tối đa 2 trong 3 yếu tố: Consistency (Tính nhất quán), Availability (Tính khả dụng) và Partition Tolerance (khả năng chịu lỗi phân vùng).

- Consistency (Tính nhất quán): Tất cả các node đều thấy dữ liệu giống nhau tại cùng một thời điểm
- Availability (Tính khả dụng): Hệ thống luôn phản hồi yêu cầu của người dùng (không trả lỗi), ngay cả khi một số node ngừng hoạt động
- Partition Tolerance (Khả năng chịu lỗi phân vùng): Hệ thống vẫn tiếp tục hoạt động bình thường ngay cả khi xảy ra sự cố mất kết nối mạng giữa các node

V. Các yếu tố quan trọng trong quản trị cơ sở dữ liệu và thiết kế hệ thống

1. Index

Index (chỉ mục) trong SQL là một cấu trúc dữ liệu đặc biệt giúp tăng tốc độ truy vấn và tìm kiếm thông tin trong cơ sở dữ liệu. Nó hoạt động tương tự như phần mục lục ở cuối một cuốn sách, cho phép hệ thống tìm ra dữ liệu ngay lập tức thay vì phải đọc toàn bộ các bản ghi. Các loại Index thường gặp bao gồm:

- Single-Column Index: Index được tạo ra trên một cột duy nhất

```
CREATE INDEX TenIndex  
ON TenBang (TenCot);
```

- Composite Index: Index được tạo trên nhiều cột (kết hợp). Rất hiệu quả cho các câu lệnh WHERE lọc qua nhiều điều kiện cùng lúc.

```
CREATE INDEX TenIndex  
ON TenBang (col1, col2);
```

- Unique Index: Đảm bảo các giá trị trong cột được index là duy nhất

Trong khi việc sử dụng Index tăng tốc độ truy xuất dữ liệu đáng kể cho các câu lệnh như SELECT, WHERE, ORDER BY, GROUP BY và JOIN, Index là một bảng tra cứu phụ nên cần thêm không gian lưu trữ. Đồng thời, mỗi khi lệnh INSERT, UPDATE hoặc DELETE được thực hiện, hệ thống sẽ phải mất thêm thời gian để cập nhật lại cấu trúc Index.

2. Transaction

Transaction trong SQL là tiến trình thực hiện một nhóm câu lệnh SQL. Các câu lệnh này được thực thi một cách tuần tự và độc lập. Một transaction tiêu chuẩn trong RDBMS luôn tuân thủ 4 tính chất ACID (Atomicity, Consistency, Isolation, Durability) như đã được kể ở trên. Các câu lệnh điều khiển Transaction bao gồm:

- **BEGIN TRANSACTION:** Đánh dấu điểm bắt đầu của một transaction
- **COMMIT:** Lưu tất cả các thay đổi vào cơ sở dữ liệu và kết thúc transaction thành công
- **ROLLBACK:** Hủy bỏ toàn bộ thay đổi thực hiện từ lúc bắt đầu transaction và khôi phục lại trạng thái dữ liệu ban đầu
- **SAVEPOINT:** Đánh dấu một điểm lưu tạm thời bên trong transaction, cho phép rollback về điểm này thay vì phải rollback toàn bộ

3. Locking

Trong SQL, Locking (khóa) là cơ chế kiểm soát truy cập dữ liệu đồng thời. Nó ngăn chặn tình trạng nhiều tiến trình ghi/đọc đè lên hoặc sửa đổi cùng một dòng dữ liệu tại cùng một thời điểm, đảm bảo tính toàn vẹn và nhất quán của cơ sở dữ liệu.

Cơ chế khóa trong SQL được áp dụng ở nhiều phạm vi khác nhau tùy thuộc vào yêu cầu của câu lệnh:

- Khóa dòng (Row Lock): Chỉ khóa một dòng dữ liệu cụ thể đang được xử lý, cho phép các tiến trình khác truy cập dòng khác trong cùng một bảng
- Khóa trang (Page Lock): Khóa một trang dữ liệu chứa nhiều dòng
- Khóa bảng (Table Lock): Khóa toàn bộ bảng dữ liệu, ngăn mọi tiến trình khác thao tác trên bảng cho đến khi giao dịch kết thúc

Các loại khóa chính bao gồm:

- Shared Lock (Khóa chia sẻ): Được sử dụng cho các thao tác đọc. Cho phép nhiều tiến trình cùng đọc dữ liệu nhưng ngăn tiến trình khác ghi/sửa đổi dữ liệu đó cho đến khi đọc xong.
- Exclusive Lock (Khóa độc quyền): Được sử dụng cho các thao tác ghi (INSERT, UPDATE, DELETE). Khóa này chỉ cho phép một tiến trình duy nhất được thao tác trên dữ liệu, tiến trình khác phải chờ.

Các hiện tượng liên quan tới khóa:

- Blocking (Chặn): Xảy ra khi một tiến trình giữ khóa độc quyền quá lâu khiến các tiến trình khác phải chờ. Việc này làm giảm hiệu suất thời gian phản hồi.
- Deadlock (Khóa chết): Hiện tượng hai hoặc nhiều tiến trình cùng giữ khóa và chờ đợi tài nguyên của nhau. Hậu quả là không tiến trình nào có thể hoàn thành, hệ thống sẽ tự động hủy (rollback) một trong các tiến trình để giải quyết.

4. Partition

Trong SQL, Partition là kỹ thuật chia dữ liệu lớn thành các phần độc lập, dễ quản lý hơn. Nó được chia thành hai mục đích chính: Partition Table (vật lý) giúp tăng hiệu suất cơ sở dữ liệu và Partition By (logic) dùng trong các hàm phân tích dữ liệu.

- Phân vùng bảng (Table Partitioning)

Đây là kỹ thuật chia một bảng dữ liệu hoặc chỉ mục lớn thành các phần vật lý nhỏ hơn (phân vùng) dựa trên giá trị của cột xác định. Việc này làm tăng tốc độ truy vấn và giúp việc bảo trì, lưu trữ từng phần diễn ra nhanh chóng, nhẹ nhàng hơn.

- PARTITION BY (Window Function)

PARTITION BY được sử dụng bên trong các hàm cửa sổ (Window Functions) như ROW_NUMBER(), RANK(), SUM(), AVG(). Mệnh đề này chia tập kết quả (result set) thành các nhóm nhỏ và thực hiện tính toán trên từng nhóm, giữ nguyên toàn bộ số lượng dòng ban đầu.

5. Sharding

Sharding trong SQL là kỹ thuật phân mảnh dữ liệu theo chiều ngang, chia một bảng lớn thành nhiều bảng nhỏ (các shard) chứa các dòng khác nhau nhưng có chung cấu trúc. Không giống như các Partition Table được lưu trong cùng một server, các shard này được lưu trữ trên các máy chủ vật lý khác nhau giúp tăng tốc độ truy vấn và mở rộng quy mô hệ thống.

6. Isolation

Isolation (Tính cô lập) là chữ cái “I” trong bộ quy tắc ACID. Đây là cơ chế cốt lõi trong SQL đảm bảo nhiều giao dịch (transaction) xảy ra cùng lúc không làm hỏng dữ liệu, mang lại kết quả giống như khi các giao dịch chạy tuần tự từng cái một. Nếu không có cơ chế cô lập hợp lý, các giao dịch chồng chéo sẽ gây ra 3 lỗi đồng thời chính:

- Dirty read (Đọc rác): Đọc được dữ liệu đang thay đổi bởi một giao dịch khác nhưng chưa được lưu (commit)
- Non-repeatable Read (Đọc không nhất quán): Cùng một câu lệnh chạy 2 lần trong một giao dịch nhưng kết quả khác nhau vì giao dịch khác đã sửa dữ liệu.
- Phantom Read (Đọc ảo): Số lượng bản ghi trả về thay đổi giữa các lần truy vấn trong cùng một giao dịch do có giao dịch khác thêm/xóa bản ghi.

Các mức độ cô lập của SQL bao gồm:

- Read Uncommitted Level (thấp nhất): Cho phép đọc những dữ liệu chưa được commit từ row. Thường được dùng trong các hệ thống đặt vé, đặt lịch khi mà một transaction được tính là đã hoàn thành cho dù chưa được thực thi xong. Có thể được sử dụng để giảm bớt tình trạng deadlock (khóa chết).
- Read Committed Level: Chỉ được đọc những dữ liệu đã được commit. Đây là mức độ phổ biến nhất.
- Repeatable Reads Level: Ở mức độ này transaction sẽ tạo một snapshot (ảnh chụp nhanh) của bất kỳ dữ liệu nào nó đọc để đảm bảo tính nhất quán (consistency) trong suốt giao dịch đó, ngay cả khi có transaction khác thực hiện các thay đổi và commit. Mức độ bảo mật này đảm bảo không xảy ra Non-repeatable Read.
- Serializable Level (cao nhất): Trong quá trình diễn ra transaction ở mức độ này không một transaction nào khác được quyền đọc hoặc ghi cho đến khi giao dịch hoàn thành. Cấp độ này giải quyết cả 3 lỗi giao dịch chồng chéo, nhưng việc chỉ có 1 query có thể chạy tại một thời điểm biến nó thành điểm nghẽn trong hệ thống.

7. Connection pooling

Connection pooling SQL là kỹ thuật lưu trữ một nhóm các kết nối cơ sở dữ liệu có sẵn. Thay vì mở và đóng kết nối vật lý cho từng truy vấn, tiêu tốn chi phí CPU và thời gian, ứng dụng sẽ mượn một kết nối từ “bể” này để thực thi câu lệnh SQL, sau đó trả lại để người khác sử dụng, giúp giảm thiểu độ trễ. Nếu không có kết nối rảnh (idle connection), pool sẽ tự tạo thêm kết nối. Sau một khoảng thời gian không có ai sử dụng (idle timeout) những kết nối thừa sẽ tự động được đóng bớt. Connection pool cũng hoạt động như một bộ phanh để kiểm soát số lượng kết nối tối đa.

Các thành phần quan trọng của pool bao gồm:

- Pool Size: Kích thước của pool, thường được cấu hình bởi hai chỉ số:
 - *Min Pool Size*: Số lượng kết nối luôn được duy trì sẵn
 - *Max Pool Size*: Số lượng kết nối pool được phép tạo ra, không vượt qua giới hạn hệ thống cho phép
- Idle Connection: Các kết nối đã sử dụng xong và đang trong trạng thái nghỉ, chờ để được sử dụng tiếp